

Test unitaire

Un **test unitaire** est un petit programme qui vérifie qu'une fonction spécifique de ton code fonctionne correctement.

Analogie :

Tu construis une voiture pièce par pièce. Avant de l'assembler, tu testes chaque pièce individuellement :

- ☐ Le moteur tourne-t-il ?
- ☐ Les freins fonctionnent-ils ?

Pourquoi c'est utile ?

Avantage	Explication
Trouver des bugs tôt	Avant que le code soit en production
Ne pas casser l'existant	Quand tu modifies du code, les tests te préviennent
Documentation vivante	Les tests montrent comment utiliser tes fonctions
Refactoriser sans peur	Tu peux réorganiser ton code en toute confiance

unittest vs pytest

Critère	unittest	pytest
Installation	Intégré à Python (pas besoin d'installer)	À installer : pip install pytest
Syntaxe	Plus verbeuse, style "Java"	Plus simple et concise
Fixtures	Plus complexe (setUp, tearDown)	Simple (@pytest.fixture)
Plugins	Peu	Très riche (beaucoup de plugins)
Quand l'utiliser ?	Projets standards, zéro dépendance	Projets modernes, tests plus simples

unittest : structure de base

Fichier à tester : calculs.py

```
def addition(a, b):  
    return a + b  
  
def soustraction(a, b):  
    return a - b  
  
def division(a, b):  
    if b == 0:  
        raise ValueError("Division par zéro")  
    return a / b
```

```
mon_projet/  
├── mon_code/  
│   ├── __init__.py  
│   └── calculs.py  
├── tests/  
│   ├── __init__.py  
│   └── test_calculs.py  
└── requirements.txt
```

test_calculs.py

```
import unittest  
from calculs import addition, soustraction, division  
  
class TestCalculs(unittest.TestCase):  
  
    def test_addition(self):  
        self.assertEqual(addition(2, 3), 5)  
        self.assertEqual(addition(-1, 1), 0)  
        self.assertEqual(addition(0, 0), 0)  
  
    def test_soustraction(self):  
        self.assertEqual(soustraction(5, 3), 2)  
        self.assertEqual(soustraction(0, 5), -5)  
  
    def test_division(self):  
        self.assertEqual(division(10, 2), 5)  
        self.assertEqual(division(7, 2), 3.5)  
  
    def test_division_par_zero(self):  
        with self.assertRaises(ValueError):  
            division(10, 0)  
  
if __name__ == "__main__":  
    unittest.main()
```

Les assertions principales

Assertion	Vérifie que...
<code>assertEqual(a, b)</code>	<code>a == b</code>
<code>assertNotEqual(a, b)</code>	<code>a != b</code>
<code>assertTrue(x)</code>	<code>x is True</code>
<code>assertFalse(x)</code>	<code>x is False</code>
<code>assertIsNone(x)</code>	<code>x is None</code>
<code>assertIsNotNone(x)</code>	<code>x is not None</code>
<code>assertIn(a, b)</code>	<code>a in b</code>
<code>assertNotIn(a, b)</code>	<code>a not in b</code>
<code>assertRaises(Erreur, func, args)</code>	La fonction lève une erreur
<code>assertAlmostEqual(a, b)</code>	<code>a ≈ b</code> (pour les flottants)

```
self.assertIn("pomme", ["pomme", "poire", "banane"])
self.assertAlmostEqual(0.1 + 0.2, 0.3, places=5)
self.assertRaises(ZeroDivisionError, lambda: 1/0)
```

unittest : setUp et tearDown

- ❑ setUp() = s'exécute avant CHAQUE test (pour préparer les données)
- ❑ tearDown() = s'exécute après CHAQUE test (pour nettoyer)

```
import unittest

class TestBaseDonnees(unittest.TestCase):

    def setUp(self):
        print("Préparation avant le test")
        self.donnees = [1, 2, 3, 4, 5]

    def tearDown(self):
        print("Nettoyage après le test")
        self.donnees = None

    def test_somme(self):
        resultat = sum(self.donnees)
        self.assertEqual(resultat, 15)

    def test_max(self):
        resultat = max(self.donnees)
        self.assertEqual(resultat, 5)

# Exécution :
# Préparation avant le test
# Nettoyage après le test
# Préparation avant le test
# Nettoyage après le test
```

pytest : installation et premier test

Fichier à tester : calculs.py

```
def addition(a, b):  
    return a + b  
  
def division(a, b):  
    if b == 0:  
        raise ValueError("Division par zéro")  
    return a / b
```

Exécuter :

```
pytest                # exécute tous les tests  
pytest -v             # version verbeuse  
pytest test_calculs.py # un seul fichier  
pytest -k "addition"  # tests contenant "addition"
```

```
pip install pytest
```

Fichier de test : test_calculs.py

```
import pytest  
from calculs import addition, division  
  
def test_addition():  
    assert addition(2, 3) == 5  
    assert addition(-1, 1) == 0  
    assert addition(0, 0) == 0  
  
def test_division():  
    assert division(10, 2) == 5  
    assert division(7, 2) == 3.5  
  
def test_division_par_zero():  
    with pytest.raises(ValueError):  
        division(10, 0)
```

Exemples complets :

```
def test_exemples():  
    # Égalité  
    assert 2 + 2 == 4  
  
    # Inégalité  
    assert 3 != 4  
  
    # Appartenance  
    assert "pomme" in ["pomme", "poire"]  
  
    # Booléen  
    assert True is True  
  
    # None  
    assert ma_variable is None  
  
    # Flottants (approximatif)  
    assert 0.1 + 0.2 == pytest.approx(0.3)  
  
    # Exception  
    with pytest.raises(ZeroDivisionError):  
        1 / 0
```

```
# unittest  
self.assertEqual(a, b)  
self.assertTrue(x)  
self.assertIn(a, b)  
  
# pytest  
assert a == b  
assert x is True  
assert a in b
```

pytest : les fixtures (préparation des données)

Les fixtures remplacent le setUp() de unittest.

Les fixtures remplacent le setUp() de unittest.

```
import pytest

@pytest.fixture
def donnees_test():
    """Cette fonction prépare des données pour les tests"""
    return [1, 2, 3, 4, 5]

@pytest.fixture
def utilisateur():
    return {"nom": "Alice", "age": 30}

# Utilisation : il suffit de mettre le nom en paramètre
def test_somme(donnees_test):
    assert sum(donnees_test) == 15

def test_max(donnees_test):
    assert max(donnees_test) == 5

def test_utilisateur(utilisateur):
    assert utilisateur["nom"] == "Alice"
    assert utilisateur["age"] == 30
```

Fixture avec nettoyage (yield) :

```
@pytest.fixture
def fichier_temp():
    with open("temp.txt", "w") as f:
        f.write("test")
    yield "temp.txt" # donne la valeur au test
    # Nettoyage après le test
    import os
    os.remove("temp.txt")

def test_lire_fichier(fichier_temp):
    with open(fichier_temp, "r") as f:
        assert f.read() == "test"
```

pytest : paramétrer les tests

Évite de répéter le même test avec des valeurs différentes :

```
import pytest

@pytest.mark.parametrize("a, b, attendu", [
    (2, 3, 5),
    (-1, 1, 0),
    (0, 0, 0),
    (100, 200, 300),
])
def test_addition(a, b, attendu):
    assert addition(a, b) == attendu

@pytest.mark.parametrize("a, b, attendu, erreur", [
    (10, 2, 5, None),
    (7, 2, 3.5, None),
    (10, 0, None, ValueError),
])
def test_division(a, b, attendu, erreur):
    if erreur:
        with pytest.raises(erreur):
            division(a, b)
    else:
        assert division(a, b) == attendu
```

Résultat : pytest exécute 3 tests pour addition, 3 tests pour division.